

Chapter 03 .scikit-learn SVM 분류

- 서포트벡터머신(SVM) 알고리즘 개념
- SVM_iris 붓꽃 분류
- 결정 경계 표현(등고선)

sklearn(사이킷런 : scikit-learn)

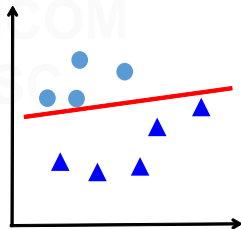
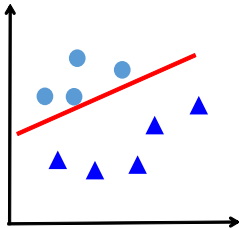
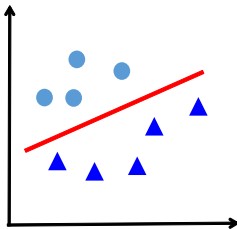
- from sklearn import svm

svm.SVC() : 결정 경계에 따른 **분류 알고리즘**

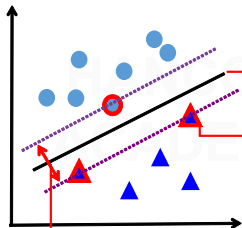
분류를 위한 기준 선(결정 경계)을 정의하는 모델

분류되지 않은 새로운 데이터는 경계의 어느 쪽에 속하는 지 확인해서 분류 수행

결정 경계를 어떻게 정의하고 계산하는지 이해하는게 중요



* 서포트 벡터 / 결정 경계 / 마진



결정 경계

: 서로 다른 분류값을 결정하는 경계

서포트 벡터

: 결정 경계선과 가장 가까이 맞는 최전방
데이터 포인트

마진 : 서포트 벡터와 결정 경계 사이의 거리

- SVM 목표 : 마진 최대로 결정 경계를 찾음
- 마진이 클수록 새로운 데이터에 대한 안정적인
분류 가능성이 높기 때문

sklearn(사이킷런 : scikit-learn)

- SVM 분류 알고리즘 장/단점

장점	단점
- 상대적으로 적은 데이터로 높은 정확도 발휘 - 예측 속도가 빠름 - 고차원 데이터 처리하기 쉬움 - 파라미터(C, gamma) 조정해서 과대/과소 적합 대처 가능	- 결정 경계선이 많이 겹칠 때는 낮은 정확도 - 커널 트릭 오사용 시 과대적합되기 쉬움 - 데이터 전처리 과정이 중요 (특성 데이터 벡터 표현) - 특성이 많을 경우 결정 경계 및 시각화 어려움

* SVM은 대표적인 이진 분류기로서 다중분류를 위해선 복잡한 설계가 필요함,
이 경우 차라리 다중분류 기능을 제공하는 랜덤포레스트등 타 모델을 사용을 추천

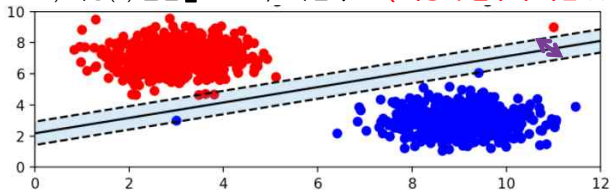
SVM 모델 하이퍼파라미터

- **C (cost)** : 얼마나 많은 이상치를 허용할지를 결정하는 값(float)
 - 낮은 C : 유연한 결정 경계를 찾음 (소프트 마진)
 - 높은 C : 타이트한 결정 경계를 찾음 (하드 마진)
- **kernel** : kernel trick에서 사용할 kernel 종류
 - kernel = "linear" : 선형 분류 SVM 경우
 - kernel = "poly" : 비선형 분류 다항식커널 SVM
 - **kernel = "rbf" : 비선형 분류 가우시안 RBF(방사기저함수) 커널 SVM**
(성능이 우수하여 현재 가장 많이 사용)
 - kernel = "sigmoid" : 시그모이드 커널 SVM
- **degree** : poly 커널 적용 시 차수를 결정하는 값(int)
- **gamma** : rbf 커널 적용 시 결정경계의 곡률을 정해주는 값(float)

* 비용(C) 파라미터 와 결정경계

이상치(outlier)를 얼마나 허용할 것인가?

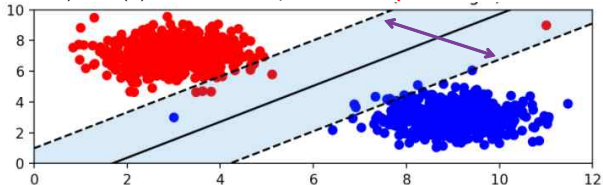
1) 비용(C) 높은 $\uparrow$ SVM / 마진 $\downarrow$ (비용이 클수록 마진 낮음)



==> 학습 에러율 감소방향 결정
경계선 생성 (하드 마진),
과대 적합(overfitting) 위험 존재

C 값이 클수록 ==> 하드 마진
(오류 허용 안 함)
오류를 허용하지 않는 범위에서 마진 최대화

2) 비용(C) 낮은 $\downarrow$ SVM / 마진 $\uparrow$ (비용이 작을수록 마진 높음)

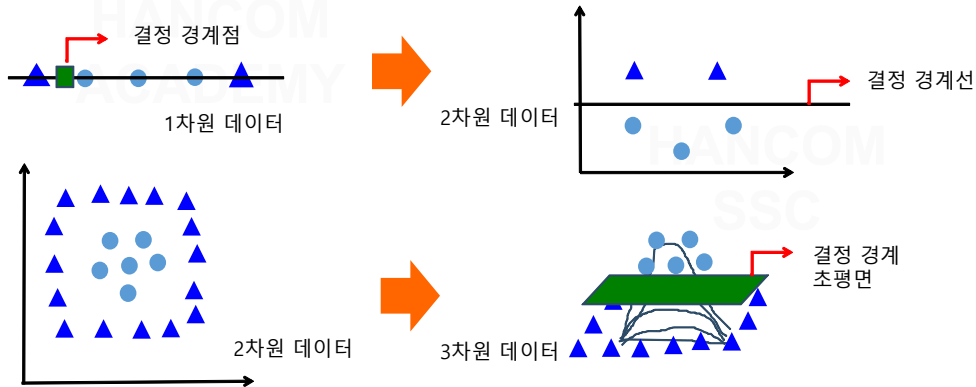


==> 학습 에러율 증가방향 결정
경계선 생성 (소프트 마진),
과소 적합(underfitting) 위험 존재

C 값이 작을수록 ==> 소프트 마진
(오류를 허용 함)
오류를 약간 허용하면서, 마진 최대화

*** 커널트릭** : 실제로 데이터를 고차원으로 보내진 않지만 보낸 것과 동일한 효과를 줘서 매우 빠른 속도로 결정 경계선을 찾는 방법

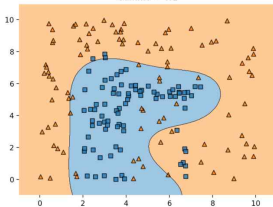
- 선형 분리가 주어진 차원에서 불가능할 경우 고차원으로 데이터를 옮기는 효과를 통해 결정 경계를 찾음
- 1차원 --> 2차원, 2차원 --> 3차원 벡터 공간으로 옮겨 결정 경계를 찾음



* 비선형 SVM 분류를 위한 커널트릭 (대표적으로 가우시안 **RBF** (Radial Basis Fuction) 커널 사용)

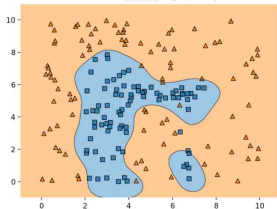
- 가우시안 RBF 커널은 데이터포인트에 적용되는 가우시안 함수의 표준편차 조정하여 결정 경계의 곡률을 조정
- 감마(gamma) : 표준편차 조정 변수
- 감마(gamma)가 커지면 데이터포인트별 허용하는 표준편차가 작아 결정 경계가 작아지면서 경계가 구부러짐 (과대적합 발생 위험)
- 감마(gamma)가 작아지면 표준편차가 커져 경계가 완만해짐 (과소적합 발생 위험)

Gamma = 0.1

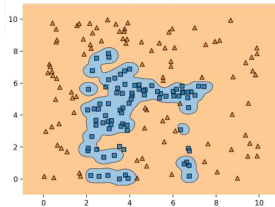


과소적합

Gamma = 1



Gamma = 10



과대적합

1) iris 붓꽃 분류 (데이터셋 로드)

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
pd.set_option('display.max_columns',100)
pd.set_option('display.width',1000)
iris = load_iris() # iris 붓꽃 데이터셋 로드
```

sepal : 꽃받침, petal : 꽃잎

(sepal_length), (sepal_width), (petal_length), (petal_width)

```
# 'data' array : array([[5.1, 3.5, 1.4, 0.2],
                        [4.9, 3. , 1.4, 0.2],
                        [4.7, 3.2, 1.3, 0.2],
                        [4.6, 3.1, 1.5, 0.2],
                        [5. , 3.6, 1.4, 0.2],
                        [5.4, 3.9, 1.7, 0.4],
                        [4.6, 3.4, 1.4, 0.3], .....])
```

```
# 'target' array : [0,0,0....1,1,1....2,2,2]
```

```
# 'target_names': array(['setosa', 'versicolor', 'virginica'])
```

```
{'data': array([[5.1, 3.5, 1.4, 0.2],
                [4.9, 3. , 1.4, 0.2],
                [4.7, 3.2, 1.3, 0.2],
                [4.6, 3.1, 1.5, 0.2],
                [5. , 3.6, 1.4, 0.2],
                [5.4, 3.9, 1.7, 0.4],
                [4.6, 3.4, 1.4, 0.3],
                .....])
```

2) iris 붓꽃 데이터셋 ---> pandas 데이터 프레임 생성

```
Iris_Data = pd.DataFrame(np.column_stack([iris['data'], iris['target']]),  
                          columns=['sepal_len','sepal_wd',  
                                  'petal_len','petal_wd','target'])  
print(Iris_Data.sample(10))
```

iris['data'] 항목과 iris['target'] 항목
np.column_stack() 이용 병합

	sepal_len	sepal_wd	petal_len	petal_wd	target
133	6.3	2.8	5.1	1.5	2.0
11	4.8	3.4	1.6	0.2	0.0
24	4.8	3.4	1.9	0.2	0.0
127	6.1	3.0	4.9	1.8	2.0
136	6.3	3.4	5.6	2.4	2.0
105	7.6	3.0	6.6	2.1	2.0
90	5.5	2.6	4.4	1.2	1.0
81	5.5	2.4	3.7	1.0	1.0
75	6.6	3.0	4.4	1.4	1.0
57	4.9	2.4	3.3	1.0	1.0

3) pandas 데이터 프레임 ---> 훈련할 샘플 및 타깃 데이터 추출

```
# petal_len, petal_wd == > 훈련할 샘플 데이터 추출  
X_petal_len_wd = Iris_Data[['petal_len', 'petal_wd']]  
print(X_petal_len_wd.sample(10))
```

```
# 레이블(클래스) 추출  
Y_target = Iris_Data[['target']]  
print(Y_target.sample(10))
```

	target
132	2.0
5	0.0
84	1.0
2	0.0
72	1.0
67	1.0
115	2.0
87	1.0
71	1.0
76	1.0

	petal_len	petal_wd
44	1.9	0.4
60	3.5	1.0
2	1.3	0.2
45	1.4	0.3
70	4.8	1.8
26	1.6	0.4
76	4.8	1.4
129	5.8	1.6
18	1.7	0.3
145	5.2	2.3

4) 훈련셋 / 테스트셋 분리 , SVM(서포트벡터머신)알고리즘 사용 훈련 및 예측

먼저 그리드서치로 최적 파라미터 c (비용) , g (감마) 값 찾아야 하나

우선 c = 1, g = 0.5 로 고정해서 테스트

c = 1

g = 0.5

훈련셋 / 테스트셋 분리

```
X_train, X_test, Y_train, Y_test = train_test_split(X_petal_len_wd, Y_target,  
                                                    test_size=0.3, random_state=0)
```

```
svm_mm = svm.SVC(C = c, kernel='rbf', gamma= g)
```

```
svm_mm.fit(X_train, Y_train.values.ravel()) # 훈련
```

```
Y_pred = svm_mm.predict(X_test) # 예측 실제 test 타깃
```

```
print(Y_test.values.ravel())
```

```
print(Y_pred)
```

```
print('accuracy : ', accuracy_score(Y_test.values.ravel(), Y_pred) )
```

강의 별첨자료
_교차검증 내용 참조

ravel() :
1차원 넘파이 배열 입력
기대

```
[2. 1. 0. 2. 0. 2. 0. 1. 1. 2. 1. 1. 1. 1. 0. 1. 1. 0. 0. 2. 1. 0. 0.  
 2. 0. 0. 1. 1. 0. 2. 1. 0. 2. 2. 1. 0. 1. 1. 1. 2. 0. 2. 0. 0.]
```

```
[2. 1. 0. 2. 0. 2. 0. 1. 1. 2. 1. 1. 1. 1. 0. 1. 1. 0. 0. 2. 1. 0. 0.  
 2. 0. 0. 1. 1. 0. 2. 1. 0. 2. 2. 1. 0. 2. 1. 1. 2. 0. 2. 0. 0.]
```

정확도

```
accuracy : 0.9777777777777777.]
```

5) 훈련셋 scatter 출력 시각화

```
lnames = iris['target_names'] # 꽃 이름 정보 ['setosa' 'versicolor' 'virginica']
markers = [ 'o', '^', 's' ] # 표시할 마커 리스트
colors = ['blue','green','red'] # 표시할 마커 색상
```

(numpy 배열, 요소 자료타입)
tuple 형태로 반환

```
# X, Y 좌표(꽃잎 길이, 꽃잎 너비) 학습(train) 데이터 scatter 출력
for i in set(Y_train['target']): # y_train = [0.0,0.0,...,1.0,1.0,...,2.0,2.0]이므로 중복 없이 {0.0, 1.0, 2.0}
```

```
    idx = np.where(Y_train['target'] == i)
```

```
    print(idx[0])
```

```
    # X_train.iloc[idx[0]] : dataframe
```

```
    # X_train.iloc[idx[0]]['petal_len'] : dataframe 의 'petal_len'컬럼 선택 : series
```

```
    # 학습데이터 타깃과 일치한 인덱스의 학습데이터 꽃잎 길이(X좌표) 구함
```

```
    #print(X_train.iloc[idx[0]]['petal_len'])
```

```
    # 학습데이터 타깃과 일치한 인덱스의 학습데이터 꽃잎 너비(Y좌표) 구함
```

```
    #print(X_train.iloc[idx[0]]['petal_wd'])
```

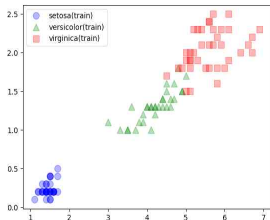
```
    # iloc => Fancy indexing 으로 X_train.iloc[idx[0]] , X_train.iloc[idx] 둘다 가능
```

```
    plt.scatter(X_train.iloc[idx[0]]['petal_len'], X_train.iloc[idx[0]]['petal_wd'],
```

```
                c = colors[int(i)], marker= markers[int(i)],
```

```
                label = lnames[int(i)]+'(train)', s=80, alpha=0.3)
```

```
[ 16 23 24 27 28 30 33 37 42 43 46 48 51 52 54 55 56 60
 61 62 65 66 68 73 75 77 78 80 89 93 98 99 100 104]
==> 해당(i 값 일치) 위치 인덱스 추출
```



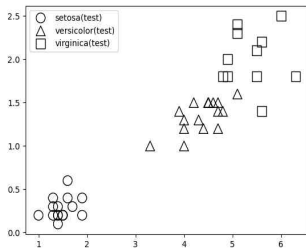
6) 테스트셋 scatter 출력 시각화

```
# X, Y 좌표(꽃잎 길이, 꽃잎 너비) 테스트(test) 데이터 scatter 출력
for i in set(Y_test['target']): # y_test = [0.0,0.0,...,1.0,1.0,...,2.0,2.0]이므로 중복 없이 { 0.0 , 1.0, 2.0 }
    idx = np.where(Y_test['target'] == i)
```

```
# 학습데이터 타깃과 일치한 인덱스의 학습데이터 꽃잎 길이(X좌표) 구함
#print(X_test.iloc[idx]['petal_len'])
# 학습데이터 타깃과 일치한 인덱스의 학습데이터 꽃잎 너비(Y좌표) 구함
#print(X_test.iloc[idx]['petal_wd'])
```

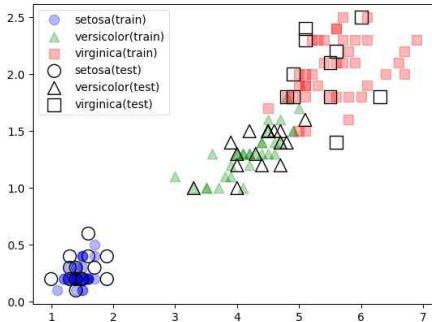
```
# X, Y 좌표(꽃잎 길이, 꽃잎 너비) 학습 데이터 scatter 출력
plt.scatter(X_test.iloc[idx]['petal_len'], X_test.iloc[idx]['petal_wd'],
            marker= markers[int(i)], label = lnames[int(i)]+'(test)',
            s=130, edgecolors='black', facecolors = 'none')
# edgecolors : 경계선 색, facecolors = 'none' : 속을 비움(색칠 x)
```

```
plt.legend(loc='best')
plt.show()
```



7) 훈련셋 / 테스트셋 scatter 동시 출력 시각화

최종 결과물
시각화



8) 등고선 시각화

등고선 출력을 위한 전체 좌표계 생성

```
X_min = X_petal_len_wd['petal_len'].min() - 0.5  
X_max = X_petal_len_wd['petal_len'].max() + 0.5  
Y_min = X_petal_len_wd['petal_wd'].min() - 0.5  
Y_max = X_petal_len_wd['petal_wd'].max() + 0.5
```

```
x1s = np.linspace(X_min, X_max, 100) # x 좌표 최소~최대 사이값 100 개 균등 분할
```

```
y1s = np.linspace(Y_min, Y_max, 100) # y 좌표 최소~최대 사이값 100 개 균등 분할
```

```
print(x1s)
```

```
print(y1s)
```

```
[0.5      0.56969697 0.63939394 0.70909091 0.77878788 0.84848485  
0.91818182 0.98787879 1.05757576 1.12727273 1.1969697 1.26666667  
1.33636364 1.40606061 1.47575758 1.54545455 1.61515152 1.68484848  
1.75454545 1.82424242 1.89393939 1.96363636 2.03333333 2.1030303  
2.17272727 2.24242424 2.31212121 2.38181818 2.45151515 2.52121212  
2.59090909 2.66060606 2.73030303 2.8      2.86969697 2.93939394  
3.00909091 3.07878788 3.14848485 3.21818182 3.28787879 3.35757576  
3.42727273 3.4969697 3.56666667 3.63636364 3.70606061 3.77575758  
3.84545455 3.91515152 3.98484848 4.05454545 4.12424242 4.19393939  
4.26363636 4.33333333 4.4030303 4.47272727 4.54242424 4.61212121  
4.68181818 4.75151515 4.82121212 4.89090909 4.96060606 5.03030303  
5.1      5.16969697 5.23939394 5.30909091 5.37878788 5.44848485  
5.51818182 5.58787879 5.65757576 5.72727273 5.7969697 5.86666667  
5.93636364 6.00606061 6.07575758 6.14545455 6.21515152 6.28484848  
6.35454545 6.42424242 6.49393939 6.56363636 6.63333333 6.7030303  
6.77272727 6.84242424 6.91212121 6.98181818 7.05151515 7.12121212  
7.19090909 7.26060606 7.33030303 7.4      ]
```

```
[-0.4      -0.36565657 -0.33131313 -0.2969697 -0.26262626 -0.22828283  
-0.19393939 -0.15959596 -0.12525253 -0.09090909 -0.05656566 -0.02222222  
0.01212121 0.04646465 0.08080808 0.11515152 0.14949495 0.18383838  
0.21818182 0.25252525 0.28686869 0.32121212 0.35555556 0.38989899  
0.42424242 0.45858586 0.49292929 0.52727273 0.56161616 0.5959596  
0.63030303 0.66464646 0.6989899 0.73333333 0.76767677 0.8020202  
0.83636364 0.87070707 0.90505051 0.93939394 0.97373737 1.00808081  
1.04242424 1.07676768 1.11111111 1.14545455 1.17979798 1.21414141  
1.24848485 1.28282828 1.31717172 1.35151515 1.38585859 1.42020202  
1.45454545 1.48888889 1.52323232 1.55757576 1.59191919 1.62626263  
1.66060606 1.69494949 1.72929293 1.76363636 1.7979798 1.83232323  
1.86666667 1.9010101 1.93535354 1.96969697 2.0040404 2.03838384  
2.07272727 2.10707071 2.14141414 2.17575758 2.21010101 2.24444444  
2.27878788 2.31313131 2.34747475 2.38181818 2.41616162 2.45050505  
2.48484848 2.51919192 2.55353535 2.58787879 2.62222222 2.65656566  
2.69090909 2.72525253 2.75959596 2.79393939 2.82828283 2.86262626  
2.8969697 2.93131313 2.96565657 3.      ]
```


9) 등고선 시각화 위한 x,y 좌표계 생성 (np.meshgrid)

그리드에 있는 모든 좌표점(행렬 데이터) 생성

x1, y1 = np.meshgrid(x1s, y1s)

y1s 데이터를 x1s 개수 만큼 오른쪽으로 펼침(중복 확장)

```
[[ -0.4      -0.4      -0.4      ...  -0.4      -0.4      -0.4      ]  
 [ -0.36565657 -0.36565657 -0.36565657 ... -0.36565657 -0.36565657 -0.36565657 ]  
 [ -0.33131313 -0.33131313 -0.33131313 ... -0.33131313 -0.33131313 -0.33131313 ]  
 ...  
 [ 2.93131313  2.93131313  2.93131313 ...  2.93131313  2.93131313  2.93131313 ]  
 [ 2.96565657  2.96565657  2.96565657 ...  2.96565657  2.96565657  2.96565657 ]  
 [ 3.         3.         3.         ...  3.         3.         3.         ]]
```

x1s 데이터를 y1s 개수 만큼 아래로 펼침(중복 확장)

```
[[ 0.5      0.56969697 0.63939394 ... 7.26060606 7.33030303 7.4      ]  
 [ 0.5      0.56969697 0.63939394 ... 7.26060606 7.33030303 7.4      ]  
 [ 0.5      0.56969697 0.63939394 ... 7.26060606 7.33030303 7.4      ]  
 ...  
 [ 0.5      0.56969697 0.63939394 ... 7.26060606 7.33030303 7.4      ]  
 [ 0.5      0.56969697 0.63939394 ... 7.26060606 7.33030303 7.4      ]  
 [ 0.5      0.56969697 0.63939394 ... 7.26060606 7.33030303 7.4      ]]
```

x축 100개, y축 100개
일 경우 모든 좌표점은
100*100 (10000)개 임

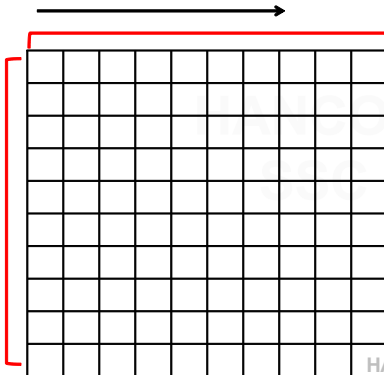
(0,99),(1,99),(2,99)....(99,99)

.....

(0,1), (1,1), (2,1),.... (99,1)

(0,0), (1,0), (2,0),.....(99,0)

100개를 총
10000개로 확장



100개를 총
10000개로 확장

10) 등고선 출력할 모든 좌표계 데이터에 대한 Z값 생성

```
# x1.ravel() : 모든 요소를 1차원 배열로 펼침(flatten)
# column_stack : 두 x,y 좌표 데이터를 합쳐 전체 좌표계 데이터 생성
xy1 = np.column_stack([x1.ravel(),y1.ravel()])
# 전체 좌표계 데이터로 예측 수행
Z = svm_mm.predict(xy1)
print(Z, Z.shape)
# Z는 1차원 배열로서 x1 또는 y1과 동일한 shape로 변형해 주어야 함( 100, 100 )
Z = Z.reshape(x1.shape)
```

[0. 0. 0. ... 2. 2. 2.] (10000,)

전체 좌표계 데이터가 어디에 속한지 예측하고 예측한 데이터로 등고선을 그림으로서 결정 경계기 표시됨



```
[[0. 0. 0. ... 2. 2. 2.]
 [0. 0. 0. ... 2. 2. 2.]
 [0. 0. 0. ... 2. 2. 2.]
 ...
 [2. 2. 2. ... 2. 2. 2.]
 [2. 2. 2. ... 2. 2. 2.]
 [2. 2. 2. ... 2. 2. 2.]] (100, 100)
```

11) 등고선 / 색 출력 (결정 경계선으로 볼수 있음)

예측 데이터를 활용한 등고선/색

cmap종류 : <https://matplotlib.org/stable/tutorials/colors/colormaps.html>

참조

Z축에 등고선 높이(Z값)값을 표현하는

Z값이 같은 점들끼리 하나의 영역으로 묶이도록 선을 그려줌

levels : Z 값 별로 각 리스트의 원소대로 cut

`plt.contour(x1, y1, Z, levels=[0.0, 1.0, 2.0], colors = 'red' )` # 등고선 표현

`plt.contourf(x1, y1, Z, cmap = plt.cm.RdYlBu, alpha = 0.3 )` # 등고색 표현

`plt.xlabel(iris['feature_names'][2])`

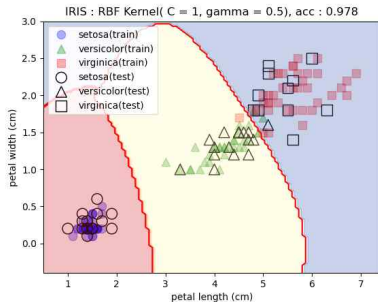
`plt.ylabel(iris['feature_names'][3])`

`plt.title('IRIS : RBF Kernel( C = {}, gamma = {}), acc :`
`{'}.format(c,g,np.round(accuracy,3)))`

`plt.legend(loc='best')`

`plt.show()`

의사결정트리_등고
선시각화 PPT 자료
참조



Thank You